When Should LLM Judges Abstain? Protocol-Stable Selective Evaluation for Trustworthy LLM-as-a-Judge

Anonymous submission

Abstract

Large language models are increasingly used as automated judges to evaluate generated text, yet these judges are prone to overconfidence, positional bias, and sensitivity to evaluation-protocol wording. We propose **CARE-Judge** (Calibrated, Abstaining, and Robust Evaluation), a selective evaluation framework that determines *when* an LLM judge's verdict should be trusted. Our central insight is *protocol-stability*: a judgment that remains invariant under semantically equivalent evaluation protocols—rubric paraphrases, response-order permutations, and repeated stochastic sampling—is significantly more likely to agree with the reference label. CARE-Judge converts multi-signal protocol-stability features into a learned correctness calibrator, then applies fixed-sequence testing with an exact Clopper–Pearson bound to accept judgments at a user-specified risk level with finite-sample guarantees. Experiments across three judge models (Qwen2.5-1.5B, DeepSeek-V4, GPT-5.5), three benchmarks (JudgeBench, TL;DR, MT-Bench Human), and 3,660 pairwise evaluations demonstrate that protocol-stability signals—particularly rubric perturbation and position-swap consistency—consistently predict judge error, with accuracy gaps of 12–54 percentage points between stable and unstable subsets. On TL;DR with DeepSeek-V4, CARE-Judge improves accepted accuracy from 65.2% to 82.2% at 11.3% coverage under a 15% risk budget. On JudgeBench with GPT-5.5, it achieves 98.2% accuracy at 42.9% coverage (α=0.05). A cost-aware cascade (Qwen-1.5B $\rightarrow$ DeepSeek $\rightarrow$ GPT-5.5) achieves 100% cost savings on TL;DR by routing all items to the free local model while maintaining 88.9% accuracy. When the judge is near-random (Qwen-1.5B on JudgeBench, 49.4%), CARE-Judge correctly abstains on all examples.

1 Introduction

Large language models (LLMs) are now widely deployed as automated judges: scoring model outputs in RLHF pipelines (Ouyang et al. 2022), ranking chatbot responses on public leaderboards (Zheng et al. 2023), filtering training data, and evaluating generation quality across diverse NLP tasks. This "LLM-as-a-judge" paradigm offers scalability that human evaluation cannot match, but it introduces a critical reliability problem: *LLM judges make systematic errors, exhibit positional biases, and vary their verdicts based on evaluation-protocol wording—all while expressing high confidence in their judgments* (Li et al. 2024; Wang et al. 2023; Wei et al. 2024).

Prior work has established that LLM judges are overconfident. Jung, Brahman, and Choi (2025) showed that standard confidence measures (predictive probability, verbalized confidence) poorly predict whether a judge's verdict agrees with human annotators, and proposed *Simulated Annotators* with cascaded selective evaluation to provide provable human-agreement guarantees. However, this approach relies on a single confidence source—simulated annotator agreement—and does not exploit the broader observation that *evaluation-protocol sensitivity itself is an error signal*.

We propose a complementary insight: **a judgment that is unstable under semantically equivalent evaluation protocols is less likely to be correct.** Concretely, if an LLM judge produces different rankings when (i) the rubric is paraphrased, (ii) the response order is swapped, or (iii) the judgment is re-sampled at nonzero temperature, the resulting verdict is less trustworthy. We call this property *protocol stability*, and we show that it yields strong, consistent predictors of judge correctness across models and datasets.

Building on this insight, we introduce **CARE-Judge** (Calibrated, Abstaining, and Robust Evaluation), a framework that:

1. Collects multi-signal protocol-stability features—rubric perturbation, position-swap consistency, self-consistency, and simulated annotator agreement—for each pairwise judgment;
2. Learns a correctness calibrator $\hat{p}(\text{correct} \mid \text{features})$ on a training split;
3. Selects a risk-controlled acceptance threshold on a disjoint calibration split using fixed-sequence testing with an exact Clopper–Pearson upper bound;
4. Reports selective risk and coverage on a held-out test split, providing a valid finite-sample guarantee.

We validate CARE-Judge across three judge models spanning three capability tiers (Qwen2.5-1.5B, DeepSeek-V4, GPT-5.5) and three benchmarks covering objective correctness (JudgeBench (Tan et al. 2024)), summarization preference (TL;DR (Stiennon et al. 2020)), and open-ended human

preference (MT-Bench Human (Zheng et al. 2023)). Our experiments yield three key findings:

- **Protocol stability predicts judge error.** Rubric perturbation and position-swap consistency produce accuracy gaps of 12–54 percentage points between stable and unstable subsets, consistently across models and datasets where the judge has at least moderate accuracy ($\geq 60\%$).
- **CARE-Judge improves accepted accuracy while controlling risk.** On TL;DR with DeepSeek-V4, CARE-Judge raises accepted accuracy from 65.2% to 82.2% at 11.3% coverage under a 15% risk budget ($\alpha = 0.15$). On TL;DR with Qwen2.5-1.5B, it improves from 88.4% to 91.8% at 64.0% coverage.
- **CARE-Judge correctly abstains when the judge is unreliable.** When Qwen2.5-1.5B (49.4% accuracy on JudgeBench) is used as a judge, CARE-Judge accepts zero examples under any reasonable risk budget, preventing the deployment of unreliable verdicts.

Our contributions are:

1. We introduce *rubric perturbation stability* as a novel, interpretable uncertainty signal for LLM-as-a-judge reliability (Section 3).
2. We formalize a multi-signal protocol-stability framework with a valid finite-sample selective-risk guarantee via exact Clopper–Pearson bounds and disjoint train/calibration/test splits (Section 3).
3. We conduct the first systematic evaluation of protocol-stability signals across multiple judge models and benchmark types, demonstrating consistent predictive power and practical accuracy improvements (Section 4).
4. We release a reproducible codebase with dataset converters, calibration utilities, and cost-aware cascade support.[1]

2 Related Work

LLM-as-a-Judge. The use of LLMs as automated evaluators has grown rapidly. Zheng et al. (2023) introduced Chatbot Arena and MT-Bench, establishing pairwise LLM judging as a scalable evaluation paradigm. Li et al. (2024) provided a comprehensive survey of LLM-based evaluation methods, identifying biases including position bias, length bias, and self-enhancement bias. Wang et al. (2023) proposed mitigating position bias via multi-turn debate and swap-and-compare. Wei et al. (2024) systematically evaluated LLM-as-a-judge prompt sensitivity, finding that rubric template diversity affects alignment scores—a finding that motivates our use of rubric perturbation as an error signal rather than merely a nuisance to be controlled.

Selective Prediction and Abstention. Selective classification (Geifman and El-Yaniv 2017; El-Yaniv and Wiener 2010) enables a predictor to abstain on uncertain inputs. Wen et al. (2025) surveyed abstention in LLMs, covering verbalized confidence, self-consistency, and retrieval-based approaches. Krishnan, Khanna, and Tickoo (2024) proposed uncertainty-calibrated fine-tuning for LLM trust, using last-layer hidden states. Our work differs by focusing on *evaluation-protocol stability* as the uncertainty source, which is specific to the judging task and does not require model internals.

Risk-Controlled Evaluation. Jung, Brahman, and Choi (2025) (Trust-or-Escalate) proposed cascaded selective evaluation for LLM judges with provable human-agreement guarantees via Simulated Annotators and fixed-sequence testing. Our framework builds on their risk-control machinery but replaces the single-signal confidence (simulated annotator agreement) with a multi-signal protocol-stability calibrator. We additionally introduce rubric perturbation as a novel signal and provide a direct comparison to their approach.

Judge Benchmarks. Tan et al. (2024) introduced JudgeBench, a benchmark of challenging response pairs with objective correctness labels across knowledge, reasoning, math, and coding. Lambert et al. (2024) proposed RewardBench for evaluating reward models. We use JudgeBench as a hard-objective benchmark and TL;DR (Stiennon et al. 2020) and MT-Bench Human (Zheng et al. 2023) as subjective-preference benchmarks, covering both correctness and preference evaluation.

Rubric-Based Evaluation. Hong et al. (2026) studied aligning LLM judges with human scoring rubrics. Anghel et al. (2025) proposed a rubric-driven multi-metric evaluation framework. These works treat rubrics as fixed evaluation criteria to be aligned with; in contrast, we treat *rubric sensitivity*—the degree to which a judge's verdict changes under semantically equivalent rubric paraphrases—as an uncertainty signal. To our knowledge, no prior work has used rubric perturbation instability as a predictor of judge error.

3 Method

CARE-Judge converts an LLM judge f into a *selective evaluator* $(\hat{f}, \hat{c})$ that either outputs a judgment $\hat{f}(x)$ or abstains, based on a confidence measure $\hat{c}(x)$ and a risk-controlled threshold $\hat{\lambda}$. The framework has three components: (1) multi-signal protocol-stability feature extraction, (2) learned correctness calibration, and (3) risk-controlled threshold selection with finite-sample guarantees.

3.1 Protocol-Stability Features

Given a pairwise evaluation item $x = (q, a_1, a_2)$ consisting of a prompt q and two candidate responses, we issue multiple judge calls under semantically equivalent evaluation protocols and measure the stability of the resulting verdicts. Let $f(x; r, \tau)$ denote the judge's output (winner $\in \{A, B\}$ and confidence $\in [0, 1]$) given rubric r and sampling temperature τ.

Rubric Perturbation Stability. We define a set of K semantically equivalent rubric variants $\{r_1, \ldots, r_K\}$ that express the same evaluation criteria in different wording (e.g., "Prefer the more correct response" vs. "Choose the answer a careful evaluator would prefer"). For each item, we collect

[1] Code and data will be released upon acceptance; an anonymized version is provided in the supplementary material.

verdicts $\{v_1, \ldots, v_K\}$ where $v_k = f(x; r_k, 0)$. We compute:

$$\text{rubric\_vote\_share} = \max_{w \in \{A,B\}} \frac{|\{k : v_k = w\}|}{K}, \quad (1)$$

$$\text{rubric\_flip} = \mathbb{1}[|\{v_1, \ldots, v_K\}| > 1], \quad (2)$$

where rubric_flip $= 1$ if any rubric variant produces a different winner. The intuition is that if the judge's verdict depends on rubric surface features rather than response content, the judgment is content-irrelevant and thus less reliable.

Position-Swap Consistency. We judge the item in both response orderings: (q, a_1, a_2) and (q, a_2, a_1). Let $v_{\text{orig}} = f(x; r_1, 0)$ and $v_{\text{swap}} = f(\text{swap}(x); r_1, 0)$, where swap reverses the response order. We map v_{swap} back to the original coordinate system and compute:

$$\text{swap\_consistency} = \mathbb{1}[\text{map}(v_{\text{swap}}) = v_{\text{orig}}]. \quad (3)$$

This probes positional bias: a judge that reverses its verdict under response reordering is unreliable.

Self-Consistency. We sample S stochastic judgments at temperature $\tau > 0$ under the base rubric and compute the majority vote share:

$$\text{self\_vote\_share} = \max_{w} \frac{|\{s : f(x; r_1, \tau_s) = w\}|}{S}. \quad (4)$$

We optionally employ *adaptive-k* early stopping: if self_vote_share $\geq \tau_{\text{stop}}$ after the minimum number of samples, we terminate early to reduce API costs.

Simulated Annotators. Following Jung, Brahman, and Choi (2025), we construct N few-shot judge personas, each conditioned on K_{shot} labeled examples drawn from the calibration pool. We compute the agreement ratio across simulated annotators as an additional confidence signal. We treat this as a *baseline signal* rather than a novel contribution.

Feature Vector. The full feature vector for item x is:

$$\boldsymbol{\phi}(x) = \big[\text{base\_conf},\ \text{rubric\_vote\_share},\ \text{rubric\_flip},\ \text{swap\_consistency},\ \text{self\_vote\_share},\ \text{sim\_vote\_share},\ \text{mean\_conf},\ \text{std\_conf},\ \text{length\_gap}\big]. \quad (5)$$

3.2 Correctness Calibration

We fit a calibrator $\hat{p} : \mathbb{R}^d \to [0, 1]$ that predicts the probability that the judge's verdict is correct given the feature vector:

$$\hat{p}(x) = P\Big(\hat{f}(x) = y \mid \boldsymbol{\phi}(x)\Big), \quad (6)$$

where y is the reference label. We support logistic regression, isotonic regression, and gradient-boosted classifiers. The calibrator is fit on a *training split* D_{train} that is disjoint from both the threshold-selection set and the evaluation set.

3.3 Risk-Controlled Threshold Selection

Given a user-specified risk tolerance $\alpha \in (0, 1)$ and error level $\delta \in (0, 1)$, we select an acceptance threshold $\hat{\lambda}$ on a *calibration split* D_{cal} (disjoint from D_{train}) such that:

$$P\Big(P\Big(\hat{f}(x) \neq y \mid \hat{c}(x) \geq \hat{\lambda}\Big) \leq \alpha\Big) \geq 1 - \delta. \quad (7)$$

We employ fixed-sequence testing (Bauer 1991): sorting calibration items by descending confidence, we sweep thresholds from high to low and select the lowest threshold $\hat{\lambda}$ whose $(1 - \delta)$ upper confidence bound on the error rate remains at most α. We use the *exact* one-sided Clopper–Pearson upper bound (Clopper and Pearson 1934):

$$\text{CP}_{\text{upper}}(e, n, \delta) = \sup\{p : P(\text{Bin}(n, p) \leq e) \geq \delta\}, \quad (8)$$

where e is the number of errors among the n accepted calibration items. This bound is computed exactly via bisection on the binomial CDF.

The selective evaluator is then applied to a *held-out test split* D_{test}:

$$(\hat{f}, \hat{c})(x) = \begin{cases} \hat{f}(x) & \text{if } \hat{c}(x) \geq \hat{\lambda}, \\ \emptyset & \text{otherwise (abstain).} \end{cases} \quad (9)$$

All reported metrics (coverage, selective risk, accepted accuracy, calibration error) are measured on D_{test}, ensuring the risk guarantee is valid.

3.4 Cost-Aware Cascade

For deployment, we arrange multiple judges $\mathcal{M} = (M_1, \ldots, M_T)$ in order of increasing cost. Each tier t has its own calibrator $\hat{p}_t$ and threshold $\hat{\lambda}_t$, calibrated independently on D_{cal}. An item is evaluated by M_1; if $\hat{p}_1(x) \geq \hat{\lambda}_1$, the verdict is accepted. Otherwise, the item is escalated to M_2, and so on. If no tier accepts, the item is abstained. The risk guarantee of (7) holds across the full cascade, ensuring that the overall selective risk does not exceed α with probability at least $1 - \delta$.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate on three benchmarks spanning objective correctness and subjective preference:

- **JudgeBench** (Tan et al. 2024): 620 pairwise comparisons with objective correctness labels across knowledge, reasoning, math, and coding.
- **TL;DR** (Stiennon et al. 2020): 300 pairwise summarization preference comparisons from the OpenAI summarization feedback dataset.
- **MT-Bench Human** (Zheng et al. 2023): 300 pairwise human preference judgments of open-ended assistant responses.

Judge Models. We use three models spanning capability tiers:

- **Qwen2.5-1.5B-Instruct**: a small local open-source model (1.5B parameters), representing a low-cost deployment scenario.
- **DeepSeek-V4 (chat mode)**: a mid-tier API model with moderate judging capability.
- **GPT-5.5**: a frontier API model representing the high-cost, high-capability tier.

Feature Collection. For each item, we collect 6 uncertainty signals via 6 judge calls: 1 base judgment (τ=0), 1 stochastic self-consistency sample (τ=0.7), 1 position-swap judgment, and 3 rubric-perturbation judgments under semantically equivalent rubric variants.

Evaluation Protocol. We use a three-way disjoint split: 40% train (fit calibrator), 30% calibration (select threshold), 30% test (report metrics). All selective metrics are measured on the held-out test split. We average over 10 random seeds. Risk parameters: α=0.15, δ=0.10. The exact Clopper–Pearson bound is used for all threshold selection.

Baselines. We compare seven methods: (1) *Raw*, which accepts all items with no abstention; (2) *Logistic fusion* (CARE-Judge), a learned calibrator over all protocol-stability features; (3) *Best single signal*, the highest-AUROC signal on the training split; (4) *Max-signal*, the maximum of the rubric, swap, and self vote shares; and the three single-signal ablations (5) *Rubric-only*, (6) *Swap-only*, and (7) *Self-only*.

4.2 Main Results

Table 1 presents the main held-out results across all model–dataset combinations.

Finding 1: Protocol-stability signals predict judge error. Across all model–dataset pairs where the judge achieves at least 60% raw accuracy, rubric perturbation and position-swap consistency produce substantial accuracy gaps between stable and unstable subsets (Table 2). The strongest single signal is position-swap consistency on TL;DR with DeepSeek-V4: 78.3% accuracy when consistent vs. 24.3% when inconsistent—a 54.0pp gap. For GPT-5.5 on TL;DR, rubric stability produces a 43.0pp gap (77.5% vs. 34.5%).

Finding 2: CARE-Judge improves accepted accuracy. On TL;DR with DeepSeek-V4, CARE-Judge improves accepted accuracy from 65.2% to 82.2% (+17.0pp) at 11.3% coverage. On TL;DR with Qwen-1.5B, it improves from 88.4% to 91.8% (+3.4pp) at 64.0% coverage. On JudgeBench with GPT-5.5, CARE-Judge improves from 91.3% to 92.9% (+1.6pp) at 64.6% coverage. At the stricter α=0.05, GPT-5.5 on JudgeBench achieves 98.2% accuracy at 42.9% coverage. The alpha-sweep (Table 3) shows a graceful risk-coverage tradeoff.

Finding 3: CARE-Judge correctly abstains on unreliable judges. When Qwen2.5-1.5B (49.4% accuracy on JudgeBench) is used as a judge, CARE-Judge accepts zero examples under any $\alpha \leq 0.30$. The protocol-stability signals also fail to discriminate on this pair (rubric gap $= -0.089$, reversed), confirming that signals require a minimum judge capability to be informative.

4.3 Ablation Study

Table 4 shows the method comparison on TL;DR with DeepSeek-V4. Logistic fusion (AUROC 0.766) outperforms all single-signal baselines. The swap-only baseline achieves the second-highest AUROC (0.714), confirming position-swap consistency as a strong individual signal, but cannot achieve nonzero coverage at α=0.15 due to its coarse binary nature.

4.4 Cross-Model Method Comparison

Table 5 compares methods across both API models. On JudgeBench with GPT-5.5, logistic fusion achieves 64.6% coverage at 92.9% accuracy (AUROC 0.815), demonstrating practical value even with a frontier model. Notably, rubric-only achieves 96.8% coverage with 92.4% accuracy on this pair, suggesting that for strong models on objective tasks, rubric stability alone is a near-perfect acceptance signal.

4.5 Model-Size Dependence

Comparing Qwen2.5-1.5B, DeepSeek-V4, and GPT-5.5 reveals that protocol-stability signals become more effective as judge capability increases, up to a point of diminishing returns. On JudgeBench, Qwen-1.5B (49.4%) produces reversed signal gaps, DeepSeek-V4 (67.4%) produces consistent 12–16pp gaps, and GPT-5.5 (91.3%) produces 12–13pp gaps with AUROC 0.815. At the frontier regime, CARE-Judge's marginal accuracy improvement is smaller (+1.6pp on JudgeBench) but maintains high coverage (64.6%), making it practically useful for deployment.

4.6 Cost-Aware Cascade

We evaluate a three-tier cascade (Qwen-1.5B $\rightarrow$ DeepSeek-V4 $\rightarrow$ GPT-5.5) with per-tier calibrated thresholds (Table 6).

On TL;DR, the cascade is highly effective: Qwen-1.5B alone handles all accepted items at $\alpha \geq 0.20$, achieving 88.9% accuracy with **100% cost savings** (zero API cost). On MT-Bench, the cascade routes 57–81% of items to the free Qwen-1.5B tier at $\alpha \geq 0.25$, saving 55–78% of cost. On JudgeBench, cheap tiers cannot meet the risk threshold, so all items escalate to GPT-5.5. This confirms that the cascade's value depends on task difficulty: easy tasks can be handled by free local models, while hard tasks require escalation to frontier models.

4.7 Cross-Dataset Transfer

We test whether a calibrator trained on one dataset transfers to another (Table 7). For GPT-5.5, a calibrator trained on TL;DR transfers to JudgeBench with 100% coverage and 91.1% accuracy (vs. 91.3% in-domain), indicating that protocol-stability signals generalize across task types for strong models. For DeepSeek-V4, TL;DR$\rightarrow$JudgeBench achieves 9.4% coverage at 97.1% accuracy. Transfer to MT-Bench is unsuccessful for both models, suggesting that open-ended human preference is a harder transfer target.

5 Conclusion

We proposed CARE-Judge, a selective evaluation framework for LLM-as-a-judge that exploits *protocol-stability signals*—particularly rubric perturbation and position-swap consistency—to determine when an LLM judge's verdict should be trusted. Our central finding is that evaluation-protocol instability is a strong, consistent predictor of judge

Table 1: Held-out selective evaluation results (10 seeds, α=0.15, δ=0.10, exact Clopper–Pearson). A coverage of 0.000 (accepted accuracy "—") is a genuine result, not a missing value: under the given risk budget the calibrated threshold accepts *no* item, i.e. CARE-Judge abstains on the entire test set because no confidence level satisfies the risk constraint. This is the intended safe behavior for unreliable judge–dataset pairs.

Model	Dataset	Raw Acc.	Coverage	Accepted Acc.	AUROC
Qwen-1.5B	JudgeBench	0.494	0.000	—	0.523
	TL;DR	0.884	0.640	0.918	0.755
	MT-Bench	0.661	0.032	0.690	0.616
DeepSeek-V4	JudgeBench	0.674	0.016	0.897	0.565
	TL;DR	0.652	0.113	0.822	0.766
	MT-Bench	0.696	0.000	—	0.617
GPT-5.5	JudgeBench	0.913	0.646	0.929	0.815
	TL;DR	0.746	0.063	0.812	0.666
	MT-Bench	0.649	0.000	—	0.557

Table 2: Signal-level accuracy: stable vs. unstable subsets. Only model–dataset pairs with $\geq$60% raw accuracy are shown.

Signal	Model	Dataset	Stable	Unstable	Gap
Rubric	Qwen-1.5B	TL;DR	0.932	0.620	+0.312
	Qwen-1.5B	MT-Bench	0.674	0.557	+0.117
	DeepSeek	JudgeBench	0.700	0.540	+0.160
	DeepSeek	TL;DR	0.689	0.538	+0.151
	DeepSeek	MT-Bench	0.712	0.558	+0.154
	GPT-5.5	TL;DR	0.775	0.345	+0.430
	GPT-5.5	JudgeBench	0.936	0.811	+0.125
Position	Qwen-1.5B	MT-Bench	0.764	0.511	+0.253
	DeepSeek	JudgeBench	0.713	0.589	+0.124
	DeepSeek	TL;DR	0.783	0.243	+0.540
	DeepSeek	MT-Bench	0.746	0.514	+0.232
	GPT-5.5	JudgeBench	0.927	0.797	+0.130
	GPT-5.5	TL;DR	0.773	0.387	+0.386
	GPT-5.5	MT-Bench	0.689	0.472	+0.217

error: when semantically equivalent rubrics or response orderings produce different verdicts, the judgment is substantially less likely to be correct. CARE-Judge converts these signals into a learned correctness calibrator with a valid finite-sample selective-risk guarantee via exact Clopper–Pearson bounds and disjoint train/calibration/test splits.

Experiments across three judge models (Qwen2.5-1.5B, DeepSeek-V4, GPT-5.5) and three benchmarks (JudgeBench, TL;DR, MT-Bench Human) demonstrate that: (i) protocol-stability signals produce 12–54pp accuracy gaps between stable and unstable subsets; (ii) CARE-Judge improves accepted accuracy by up to 17pp while maintaining useful coverage; (iii) the framework correctly abstains entirely when the judge is near-random, preventing unreliable deployment; (iv) a cost-aware cascade routes easy items to a free local model, achieving up to 100% API-cost savings on TL;DR while retaining 88.9% accuracy; and (v) calibrators transfer across datasets for strong judges (TL;DR$\rightarrow$JudgeBench: 100% coverage at 91.1% accuracy for GPT-5.5).

Limitations. Protocol-stability signals require a minimum judge capability: when the judge's raw accuracy is near chance (e.g., Qwen-1.5B on JudgeBench), the signals lose discriminative power. Additionally, our current evaluation uses 6–10 judge calls per item, which increases evaluation cost relative to single-call judging; the adaptive-k mechanism and the cost-aware cascade partially mitigate this. Finally, cross-dataset transfer to open-ended human-preference tasks (MT-Bench) remains difficult, indicating that the calibrator's transferability depends on task type.

Future Work. Promising directions include: (1) extending protocol-stability signals to pointwise (non-pairwise) evaluation, (2) developing domain-aware signal selection (e.g., position-swap may be more informative for subjective preference, rubric perturbation for factual correctness), and (3) integrating protocol stability into active learning for human annotation triage.

Table 3: Alpha-sweep risk-coverage tradeoff (logistic fusion, 10 seeds, held-out test). Each cell reports coverage / accepted accuracy. "–" denotes safe abstention (zero coverage), as in Table 1.

Model	Dataset	α=0.05	α=0.08	α=0.10	α=0.12	α=0.15	α=0.20	α=0.25
Qwen-1.5B	TL;DR	.06/.92	.25/.96	.47/.95	.50/.94	.64/.92	.98/.90	.92/.90
	MT-Bench	–/–	–/–	–/–	–/–	.03/.69	.03/.69	.10/.71
DeepSeek-V4	JudgeBench	–/–	–/–	.01/.95	.01/.95	.02/.90	.03/.86	.07/.74
	TL;DR	–/–	–/–	–/–	.02/.81	.11/.82	.21/.81	.41/.79
	MT-Bench	–/–	–/–	–/–	–/–	–/–	.06/.70	.14/.75
GPT-5.5	JudgeBench	.43/.98	.75/.96	.83/.95	.80/.93	.65/.93	.99/.92	.99/.92
	TL;DR	–/–	–/–	–/–	.02/.86	.06/.81	.16/.82	.38/.80
	MT-Bench	–/–	–/–	–/–	–/–	–/–	–/–	.02/.62

Table 4: Ablation: method comparison on TL;DR with DeepSeek-V4 (10 seeds, held-out, α=0.15). Coverage 0.000 denotes safe abstention (Table 1).

Method	AUROC	Coverage	Accepted Acc.
Raw (no abstention)	0.500	1.000	0.652
Logistic fusion (CARE)	**0.766**	**0.113**	**0.822**
Best single signal	0.672	0.000	—
Max signal	0.522	0.000	—
Rubric only	0.541	0.000	—
Swap only	0.714	0.000	—
Self only	0.533	0.000	—

Table 5: Method comparison: coverage / AUROC (α=0.15, 10 seeds). JB = JudgeBench, MT-B = MT-Bench.

	DeepSeek-V4			GPT-5.5		
Method	JB	TL;DR	MT-B	JB	TL;DR	MT-B
Raw	1.0/.50	1.0/.50	1.0/.50	1.0/.50	1.0/.50	1.0/.50
Fusion	.02/.57	.11/.77	.00/.62	.65/.82	.06/.67	.00/.56
Best	.06/.60	.00/.67	.00/.64	.99/.82	.18/.69	.00/.60
Rubric	.00/.53	.00/.54	.00/.55	.97/.65	.00/.61	.00/.52
Swap	.00/.57	.00/.71	.00/.59	.94/.57	.00/.59	.00/.53
Self	.00/.52	.00/.53	.00/.52	.98/.52	.00/.54	.00/.50

Table 6: Cost-aware cascade (Qwen-1.5B → DeepSeek → GPT-5.5). Columns Qwen/DS/GPT-5.5 give the number of items resolved at each tier; savings are relative to a GPT-5.5-only judge.

Dataset	α	Cov.	Acc.	Qwen	DS	GPT-5.5	Save
TL;DR	0.15	0.967	0.902	174	0	0	96%
	0.20	1.000	0.889	180	0	0	100%
	0.30	1.000	0.889	180	0	0	100%
MT-Bench	0.20	0.433	0.705	68	0	10	25%
	0.25	0.667	0.692	103	11	6	55%
	0.30	0.850	0.641	146	2	5	78%
JudgeBench	0.25	0.997	0.919	0	0	371	−20%
	0.30	0.997	0.898	0	64	307	−3%

Table 7: Cross-dataset transfer (α=0.15, held-out test). Zero coverage denotes safe abstention, as in Table 1.

Model	Source → Target	Cov.	Acc.	N
DeepSeek	TL;DR → JudgeBench	0.094	0.971	372
DeepSeek	MT-Bench → JudgeBench	0.091	0.824	372
GPT-5.5	TL;DR → JudgeBench	1.000	0.911	372
GPT-5.5	MT-Bench → JudgeBench	1.000	0.911	372
(all → MT-Bench)		0.000	—	—

References

Anghel, C.; Anghel, A. A.; Pecheanu, E.; Crăciun, M.; Cocu, A.; and Niculita, C. 2025. PEARL: A Rubric-Driven Multi-Metric Framework for LLM Evaluation. *Information*, 16(11).

Bauer, P. 1991. A Note on Multiple Testing Procedures in Structural Break Tests. *Journal of Statistical Planning and Inference*.

Clopper, C. J.; and Pearson, E. S. 1934. The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. *Biometrika*, 26(4): 404–413.

El-Yaniv, R.; and Wiener, Y. 2010. On the Foundations of Noise-free Selective Classification. *JMLR*.

Geifman, Y.; and El-Yaniv, R. 2017. Selective Classification for Deep Neural Networks. In *NeurIPS*.

Hong, Y.; Yao, H.; Shen, B.; Xu, W.; Wei, H.; and Dong, Y. 2026. From Rubrics to Reliable Scores: Evidence-Grounded Text Evaluation with LLM Judges. In *arXiv:2601.08654*.

Jung, J.; Brahman, F.; and Choi, Y. 2025. Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement. In *ICLR*.

Krishnan, R.; Khanna, P.; and Tickoo, O. 2024. Enhancing Trust in Large Language Models via Uncertainty-Calibrated Fine-Tuning. In *arXiv:2412.02904*.

Lambert, N.; Pyatkin, V.; Morrison, J.; et al. 2024. RewardBench: Evaluating Reward Models for Language Modeling. In *arXiv:2403.13787*.

Li, H.; Dong, Q.; Chen, J.; Su, H.; Zhou, Y.; Ai, Q.; Ye, Z.; and Liu, Y. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. *arXiv preprint arXiv:2412.05579*.

Ouyang, L.; Wu, J.; Jiang, X.; et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *NeurIPS*.

Stiennon, N.; Long, L.; Hou, Y.; Wu, J.; Ziegler, D.; Radford, R.; Amodei, D.; and Christiano, P. 2020. Learning to Summarize with Human Feedback. *NeurIPS*.

Tan, S.; Zhuang, S.; Montgomery, K. L.; Tang, W.; Cuadron, A.; and Wang, C. 2024. JudgeBench: A Benchmark for Evaluating LLM-Based Judges. In *arXiv:2410.12784*.

Wang, P.; Li, L.; Chen, L.; Zhu, D.; Lin, B.; Cao, Y.; Liu, Q.; Liu, T.; and Sui, Z. 2023. Large Language Models are not Fair Evaluators. In *arXiv:2305.17926*.

Wei, H.; He, S.; Xia, T.; Liu, F.; Wong, A.; Lin, J.; and Han, M. 2024. Systematic Evaluation of LLM-as-a-Judge in LLM Alignment Tasks: Explainable Metrics and Diverse Prompt Templates. In *arXiv:2408.13006*.

Wen, B.; Yao, J.; Feng, S.; Xu, C.; Tsvetkov, Y.; Howe, B.; and Wang, L. L. 2025. Know Your Limits: A Survey of Abstention in Large Language Models. *TACL*, 13: 529–556.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*.